

Project Lionheart: The Story So Far...

The day after the mini-quality attribute workshop (see [Activity 7, Mini-Quality Attribute Workshop, on page 210](#)), our team gathers to discuss architecture options. We knew we were building a data-driven web application, but beyond that we didn't have much else to go on. What overall patterns would we use to organize the system? What technologies will we use for the web application? How will the code be deployed and hosted? How will the code be organized?

You review the constraints with the team as well as some of the more interesting functional requirements. Looking at the influential functional requirements you point out that we'll need a database and a search engine. Most of the team has prior experience with MySQL, so you guide the team into choosing that for the database. After the review meeting you create a task in the backlog to explore search engine technologies and select one.

Before we start coding, we still needed to make some basic decisions about organizing the code and how the coarse-grained components in the system will come together. Using the think-do-check cycle as a guide, you take a moment to *think* and plan our next steps.

You start writing open questions on the whiteboard and ask the team to share their concerns too. Soon there is a long list of questions and risks on the whiteboard. You shift to the *do* part of our design process. "We'll cover more ground if we divide and conquer (see [Activity 15, Divide and Conquer, on page 239](#)) to find answers," you say. "Half the team will explore patterns that let us achieve our quality attributes. The other half will explore our technology questions." To *check*, you decide that everyone will reconvene in one week to share findings and make decisions.

Next Up

Choosing structures is easy. Choosing appropriate structures is difficult. In this chapter, we solved some of the mysteries behind the thought process that goes into architectural decision making. Accept the constraints. Find the interesting functional requirements and ensure the architecture can achieve them. Explore patterns to help promote desired quality attributes. Make decisions at the right time and always promote changeability when practical.

Making design decisions is never easy, but it becomes easier with experience. In the next chapter, you'll bootstrap your design experience by learning some common architecture patterns.

[Understand](#)[Explore](#)[Evaluate](#)[Make](#)

CHAPTER 7

Create a Foundation with Patterns

For hundreds of years, engineers have captured solutions to common problems as reusable patterns. Software engineers follow this tradition as well. Seasoned software architects know many patterns. When facing a new problem, experienced architects explore their catalog of design patterns to find likely solutions before attempting to design something new. Once they've identified a suitable pattern, they'll fill in details for the particular problem at hand and adapt it to meet their current needs.

Every software system employs a small number of thematic patterns as a foundation on which we make all other design decisions. Using patterns is like having the greatest minds in software architecture on your team. When you use a pattern, you gain the benefits of others' wisdom without investing much work on your own.

Hundreds of architecture design patterns are available that span a variety of domains and contexts. In this chapter, you'll explore some of the most common architecture patterns and briefly discuss how to adapt those general patterns to meet specific needs.

What Is an Architecture Pattern?

Many of the technical problems software architects face are not new. As a broader software architecture community, we've been building scalable, maintainable, reliable, highly available, testable software systems across a variety of technical domains for decades. Apart from a small handful of emerging problem areas, many of today's software design problems have known solutions. Patterns describe these known solutions.

An *architecture pattern* is a reusable solution to a specific problem. Software architecture patterns show how to promote specific quality attributes by using

a specific combination of structures. Choose the right patterns for our problem, and we can avoid nasty traps that may otherwise cause trouble had we attempted to design the architecture from scratch.

Patterns have many other benefits. Since many patterns are widely known, we get communication bonuses by using them. If a picture is worth a thousand words, a pattern is worth a thousand pictures. Popular patterns are baked into frameworks and platforms, making it easier to adopt them.

Let's explore some of the more common architecture patterns in use today. The mini-catalog here is far from a complete list. More information about each pattern can be found easily on the web and in existing architecture literature.



Joe asks:

What Is the Difference between a Design Pattern and an Architecture Pattern?

Design patterns are an essential design tool for all designers regardless of design discipline or granularity of abstraction. You'll find design patterns for user experience, testing, database design, and even engineering processes, in addition to programming, software architecture, and enterprise architecture. All design patterns have a place in modern software development.

Architecture patterns differ from programming design patterns, such as those cataloged by the Gang of Four in [Design Patterns: Elements of Reusable Object-Oriented Software \[GHJV95\]](#), by the types of problems they aim to solve. The Gang of Four's design patterns shows how to organize object-oriented programs to promote reusability and maintainability. Architecture patterns define solutions for a variety of quality attributes scenarios—design time, runtime, and conceptual—and often deal with multiple components of a software system. The scope is broader in an architecture pattern regarding both the quality attributes and the granularity of abstractions in play.

In practice, distinguishing programming design patterns from architecture design patterns isn't so important. After all, one person's architecture might be another person's detailed design.

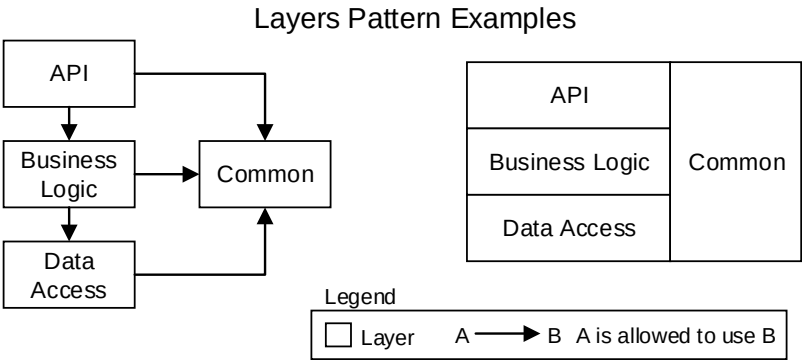
Layers Pattern

The *layers pattern* is one of the most used (and abused) architecture patterns. Most software systems have multiple contributors. Partitioning code into distinct and independent layers organized around a specific set of related concerns enables developers to work together better. Layers promote decreased coupling between the layers and higher cohesion within, which promotes

maintainability. Use layers any time you need to change code modules independently of one another.

Category	Module
Elements	<i>Layer</i> —group of functionally cohesive modules.
Relations	<i>Allowed to use</i> —indicates which layers may use modules within another layer.
Rules for Use	Every module must be allocated to one and only one layer. Layers above are allowed to use layers below, but this relation only goes one way. The <i>allowed to use</i> relation can be limited so the current layer may only use the layer immediately below it. Cyclical dependencies are not permitted.
Strengths	Promotes maintainability, portability, reusability, testability, design time modifiability. Conceptually simple to implement. Layers can be made visible in the code.
Weaknesses	Each layer introduces additional abstractions between the highest layers and the lowest. These additional abstractions increase complexity and may harm performance. Too many layers or leaky abstractions can make development painful for programmers.

Here are examples of layered diagrams. The diagram on the left explicitly shows the *allowed to use* relation, whereas the diagram on the right implies relations among elements.



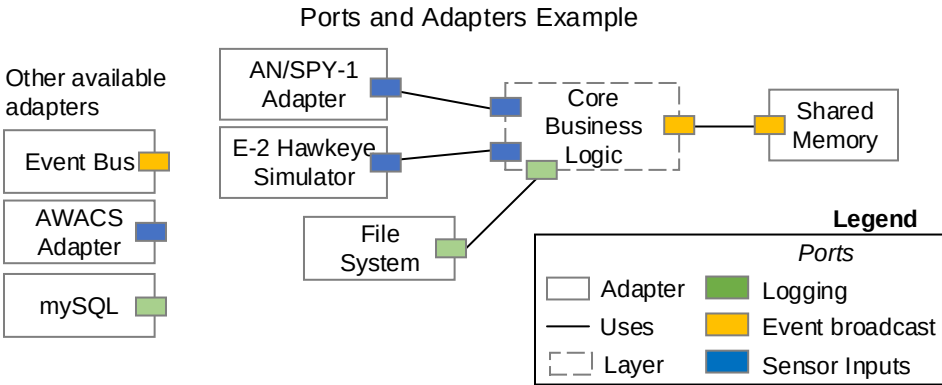
There many variants of the layered pattern. No matter how it's drawn or how many layers are involved, the elements, relations, and rules for their use are the same.

Ports and Adapters Pattern

The *ports and adapters pattern* isolates core business logic so it can be used in a variety of contexts and tested in isolation from components that provide data and events. At runtime, pluggable adapters for specific input sources can be injected into the core business logic to provide access to events and data. Adapters can be swapped at build-time or runtime to create different configurations of the software system. Use this pattern when the system must support multiple input devices or when there is a risk that input devices could change.

This pattern was initially described by Alistair Cockburn under the name *Hexagonal Architecture*.¹ See the [table on page 83](#).

Here is an example of a ports-and-adapters diagram. In this example, radar simulators can be swapped for adapters to real radar systems without changing the core business logic. The logging and communication bus can also be exchanged depending on the situation.



1. <http://alistair.cockburn.us/Hexagonal+architecture>

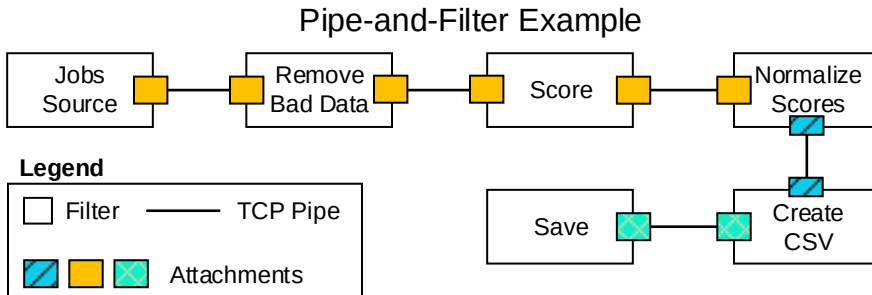
Category	Module or Component & Connector
Elements	<i>Layer</i>—Contains domain or business logic that has no knowledge of where data or events it uses originates. <i>Port</i>—Describes the interface between a layer and an adapter. Ports allow layers to be decoupled from concrete adapters. <i>Adapter</i>—Code that interacts with external data sources, devices, or other components that layers can use to gain access to data or events.
Relations	<i>Exposes</i>—Indicates the ports available from a specific layer. <i>Implements</i>—Describes the ports which constrain an adapter. <i>Injects</i>—Indicates which adapters will be available to a given layer.
Rules for Use	Layers usually but are not required to expose ports. Layers without ports are sometimes referred to as <i>inner layers</i>. Adapters may satisfy the constraints of one or more ports. An adapter may only be injected into a port when that adapter implements the interface required by that port. Depending on the mechanisms used to realize elements and relations, the pattern may refer to design-time or run-time interactions. Be clear and consistent in your models whether you are showing module or C&C structures.
Strengths	Promotes testability, maintainability, and modifiability. Different teams can work on different layers or adapters.
Weaknesses	Mechanisms must be developed to select adapters used at runtime. Runtime qualities such as security and reliability are decided by the adapters. Third-party adapters should be selected with care.

Pipe-and-Filter Pattern

With the *pipe-and-filter pattern*, each component called a filter is responsible for a single transformation or data operation. Data is streamed from one filter to the next as quickly as possible, and data operations occur in parallel. Loosely coupled filters can be reused and combined in different ways to create new pipelines.

The pipe-and-filter pattern is prevalent in data analysis and data transformation use cases. If you've ever piped Unix commands together in a terminal window, then you have firsthand experience with the pipe-and-filter pattern. See the [table on page 85](#).

Here is an example diagram of the pipe-and-filter pattern:



The *batch sequential pattern* is similar to pipe-and-filter but has one major difference. The stages of a batch-sequential system operate in turn, one at a time instead of in parallel like in a pipe-and-filter system. Instead of streaming data from one stage to the next, batch sequential systems usually write all data to disk for the next stage in the sequence to read.

Category	Component & Connector
Elements	<i>Filter</i>—A component that reads data, transforms it, then writes out the transformed data. Filters may begin processing data as soon as it is read. Filters must define expected inputs and produced outputs. <i>Pipe</i>—A connector, which transports data from one filter to the next, preserving data order. Pipes have a single input and output, and do not alter the data in transit. Some variants of this pattern also include <i>source</i> and <i>sink</i> elements. The former only produces data whereas the latter only receives it.
Relations	<i>Attachment</i> —Connects the output of one filter with the input of another by way of a pipe.
Rules for Use	Pipes can only connect filters with compatible inputs and outputs. Filters should be completely independent of one another and have no knowledge of upstream or downstream filters.
Strengths	Promotes performance, reusability, and modifiability.
Weaknesses	Pipe-and-filter systems are not interactive and cannot include a user interface without modifying the pattern. Reliability is not specifically promoted by the pattern but can be designed in by introducing filters to handle error cases. A naive implementation can harm performance because having many filters running in parallel can be computationally expensive.

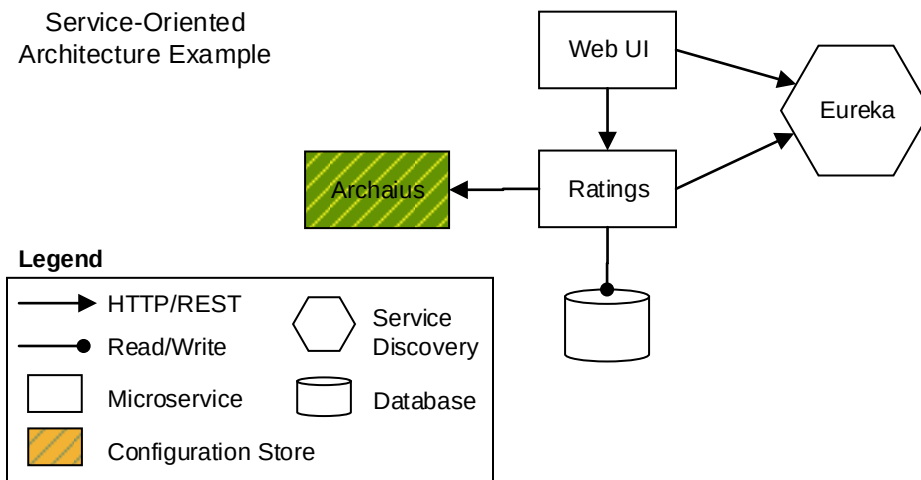
Service-Oriented Architecture Pattern

In a *service-oriented architecture*, independent components are implemented as services, which provide specific functionality. Services are combined at runtime to define the software system's behavior. For this to work, service consumers must be able to locate and use services without knowing about implementation details behind the services they use.

Service-oriented architectures (SOA) can be implemented in many ways. Traditional SOA relies heavily on message buses and communication via SOAP. Modern SOA encourages the use of fine-grained microservices connected by lightweight message protocols such as HTTP.

Complex organizations will often turn to SOA to design large software systems in which different departments own different pieces of the system. SOA allows each department to work independently within their area of expertise and hide information systems but also provides broad access to those subsystems without compromising design integrity. See the [table on page 87](#).

Here is an oversimplified example diagram showing a single view of a service-oriented system. Service-oriented architectures are complicated and involve many architectural components. This diagram shows two services attached to the service registry. Services must check the registry to look up connection information for other services they want to call.



Category	Component & Connector
Elements	<i>Service</i>—An independently deployable unit, which provides functionality behind a well-defined interface. <i>Service registry</i>—List of all available services, used by a service to discover other services to use. <i>Message system</i>—The specific element type depends on the flavor of SOA and other design decisions around service communication. Some examples include SOAP, REST (representational state transfer, usually over HTTP), gRPC,² and asynchronous messaging.
Relations	Varies depending on the constraints in the SOA system. With the smart endpoints, dumb pipes approach popularized by Netflix, <i>calls</i> might be the only relation. If your SOA system uses asynchronous messaging, <i>publish</i> and <i>subscribe</i> might be the relations in play.
Rules for Use	Services have no knowledge about the implementation details of the services they use. Services must discover other services via an external component, either a service registry or message bus in the case of asynchronous message passing.
Strengths	Promotes interoperability, reusability, and scalability. This is a well-studied pattern with many, many subpatterns defined.
Weaknesses	SOA systems are distributed systems and include all the complexity that comes with distributed computation. SOA systems have many parts and can be complicated to assemble. Properties that can be handled easily at design time in other patterns are a runtime concern with SOA. For example, it's impossible to know the version of an SOA system without knowing what services were running at a snapshot in time. Availability, reliability, and performance are inhibited by this pattern.

2. <http://www.grpc.io/>

Avoid Architectural Mismatch

Architectural mismatch, described in [Architectural Mismatch: Why Reuse Is So Hard \[GAO95\]](#), is a phenomenon that occurs when the assumptions made about how a component will be used conflicts with that component's current use. In the best case, a system experiencing architectural mismatch will be difficult to develop and maintain. In the worst case, key quality attributes will be unattainable.

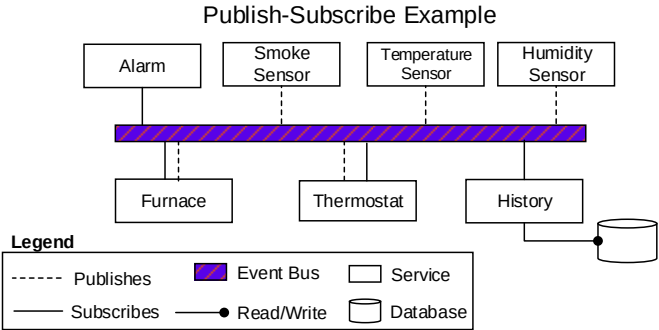
Architectural mismatch can occur at the conceptual level when a selected pattern is in direct tension with high-priority quality attributes. For example, if performance is the number one quality attribute, then selecting service-oriented architecture might be a mismatch. Picking the wrong pattern can significantly harm the architecture's ability to promote required properties.

Architectural mismatch can also occur when the technology selected to implement the system does not align with assumptions laid out in the architecture. For example, if the architecture describes a publish-subscribe pattern, using a relational database as the primary mechanism for message passing would undermine the properties promoted by the publish-subscribe pattern. When a mismatch occurs on my team, we will talk about how we are *fighting the framework*. To avoid creating a mismatch, choose technologies that match the assumptions made in the architecture.

Publish-Subscribe Pattern

In the *publish-subscribe pattern*, producers and consumers exist independently and unaware of one another. Numerous consumers subscribe to events published by various producers. Producers and consumers communicate indirectly via an event bus, which is responsible for connecting published events with interested subscribers. The choice of event bus technology greatly influences the systems properties. Choose the publish-subscribe pattern when multiple, independent components need access to the same information. See the [table on page 89](#).

Here is an example diagram of a publish-subscribe system:



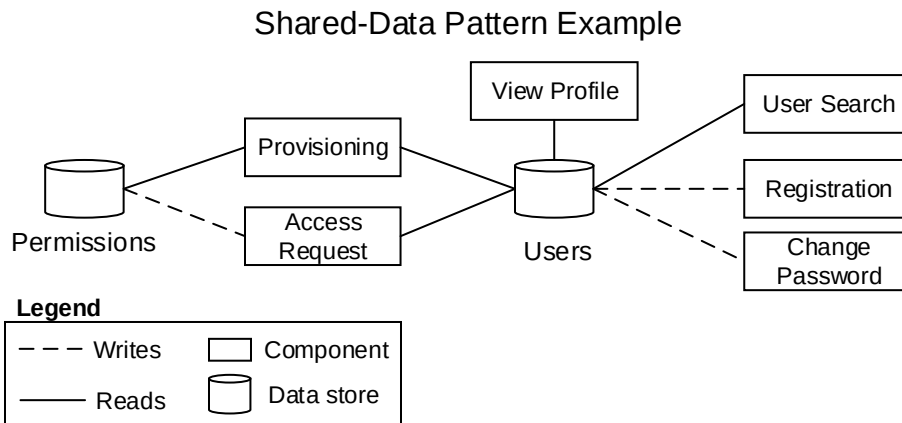
Category	Component & Connector
Elements	<i>Publisher</i>—any component that publishes an event. Specific events published should be described in design documentation. <i>Subscriber</i>—any component that subscribes to an event. <i>Event Bus</i>—responsible for registering component subscriptions and delivering published events. The properties promoted by the event bus vary on the specific technology and its configuration.
Relations	<i>Publish</i>—Indicates that a component publishes events to the event bus. <i>Subscribe</i>—Indicates that a component registers an event subscription.
Rules for Use	All communications in this pattern take place via the event bus. As such all components, must be connected to the bus. Components may be both a publisher and subscriber.
Strengths	Promotes extensibility, reusability, and testability. Depending on the selection of event bus and how it is configured, availability, reliability, and scalability might also be promoted.
Weaknesses	It is difficult to reason about performance in publish-subscribe systems given the independent, asynchronous nature of component communication. The choice of event bus is ultimately responsible for making or breaking a publish-subscribe system. Choose your event bus with care and learn everything you can about how to use it.

Most publish-subscribe systems define an *event specification*, which describes events to which components may subscribe. This document also describes event formats as well as the components responsible for publishing events.

Shared-Data Pattern

In the *shared-data pattern*, multiple components access data through a common data store. No single component is entirely responsible for the data or data store. This pattern is particularly useful when multiple components require a large amount of data. With shared-data systems, data and the data source is the primary medium of interaction. Compare this to event-based systems in which components communicate via procedure calls or message passing. See the [table on page 91](#).

The following diagram shows an example of a shared-data pattern.



The shared-data pattern blends well with other patterns. Many large information systems will use this pattern somewhere within the architecture.

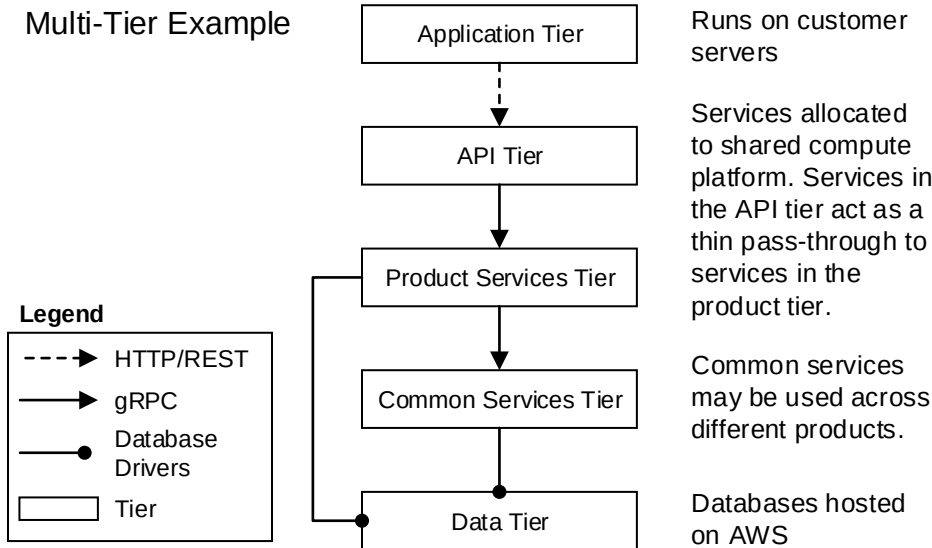
Category	Component & Connector
Elements	<i>Data store</i>—holds the data shared among accessors. Choice of the data store and the constraints placed on it determine the quality attributes achieved with the pattern. <i>Data accessor component</i>—components that use the data in some way.
Relations	<i>Reads</i>—indicates that a data accessor component may read data from the shared-data store. Some read relations might require specific protocols or place limits on the amount or types of data that can be read. <i>Writes</i>—indicates that a data accessor component writes data to the shared-data store. Write relations can be transactional, throttled, protected, or otherwise constrained in a variety of ways.
Rules for Use	Only data accessors may interact with the shared-data store.
Strengths	Promotes reliability via data consistency, security, and privacy. Scalability and availability are also promoted when the data store is tuned well and data accessors are thoughtfully partitioned.
Weaknesses	The shared-data store creates a single point of failure, which can harm availability and performance. Maintainability can be harmed if the data store changes, since all data accessors may be required to change as well. This pattern is simple to implement and prone to abuse. Sharing data can solve many problems, but depending on the context other architecture patterns could be a better fit.

Multi-Tier Pattern

In the *multi-tier pattern*, runtime structures are organized into logical groups. These logical groups may be allocated to specific physical components, such as a server or cloud platform. The multi-tier pattern is conceptually similar to the layers pattern. Layers are a module structure and deal with design-time elements, whereas tiers are either a component and connector or allocation structure and deal with runtime elements.

Any system in which components will live on different platforms or hardware will benefit by thinking about the tiers at play and which components reside on different tiers. See the [table on page 93](#).

Here is a diagram depicting the multi-tier pattern. In this example, components in the application tier are allocated to the customer's servers. Components in the middle tiers are allocated to a common platform but have different functional responsibilities. Components in the data tier are hosted on a different cloud platform and may only include databases.



Category	Component & Connector or Allocation
Elements	<i>Tier</i> —a logical grouping of runtime components. There are many ways to partition tiers. Some examples include functional responsibilities, compute platforms, team responsibilities, communication mechanisms, security requirements, and data access.
Relations	<i>Belongs to</i> —used to group components into a tier. <i>Communicates with</i> —shows how tiers or the components within interact with one another. This relation may be specialized to include information about protocols and communication constraints. <i>Allowed to communicate with</i> —indicates which tiers may communicate with components in other tiers. <i>Allocated to</i> —maps tiers to physical compute platforms.
Rules for Use	A component may belong to only one tier. Components within a tier are only allowed to communicate with other components within the same. Additional constraints describing tier communication can enhance reasoning and improve maintainability. A common approach is to allow communication only among adjacent tiers.
Strengths	Promotes security, performance, availability, maintainability, modifiability. Can be used to reason about costs and deployment.
Weaknesses	As a runtime construct, tiers can be difficult to enforce in large systems. Systems with many tiers can inhibit performance and maintainability.

Center of Competence Pattern

In the *center of competence pattern*, a team of experts is charged with defining patterns, establishing best practices, developing support tools, and providing education for a subset of the architecture. The center of competence (CoC) is not expected to build and deliver this part of the system but rather help other teams excel in their day-to-day development work. CoC teams can be organized around technologies, use cases, patterns, and high-risk areas.

Creating a CoC will make it easier for development teams to implement patterns and technologies we want in the architecture. Since the CoC is a support group, its primary goal is to increase development speed and improve the overall quality of the software system.

Category	Allocation
Elements	<i>CoC Team</i> —group of developers and architects. <i>Responsibility Area</i> —subset of the architecture. Can be a pattern, technology, or use case.
Relations	<i>Responsible for</i> —connects a CoC team with their responsibility areas.
Rules for Use	Typically a CoC is responsible for only one type of technology or use case.
Strengths	Promotes reusability and scalability of experts. Greater access to experts and reusable assets can positively influence many quality attributes, including security, scalability, performance, reliability, and maintainability.
Weaknesses	Centers of competence create pockets of expert knowledge, which can be easily disrupted by turnover. Weak CoCs can create confusion and slow down development.

Here is an example of how one company, consisting of a few hundred developers, organized center of competence teams.

CoC Team	Responsibility Area
Job Scheduling Use Case	Develop a framework for the job scheduling use case and create tools so teams can instantiate the framework on clusters themselves.
Performance	Consult with teams about load and performance testing, provide tools for testing and data collection, collect and organize data sets and other testing assets.
Database Technologies	Consult with teams to select supported database technologies appropriate to use case, maintain tools for provisioning databases, create or distribute training materials.
Core Platform	Maintain common container management system, provide supported Docker base images, create tools for day-to-day tasks such as log aggregation and health checks.

Open Source Contribution Pattern

In the *open source contribution pattern*, teams are given responsibility for developing specific architectural components but are not expected to be the only contributing developers. When this pattern works well, a team will act as a *benevolent dictator* over their components, reviewing submitted changes to a component for quality and conceptual integrity. This pattern allows for limited centralized control across the architecture. Use this pattern when there are experts available from multiple development teams or when there is a common dependency on specific components.

For this pattern to be successful, teams must know they are responsible for specific components and have a firm understanding of where their components fit within the larger context. Provide only the owning team with write access to enforce architectural responsibility for the component. All other teams should have the ability to submit changes for review. Create style guides, design for testability, and impose constraints on technologies and build platforms, to make it easier for developers to contribute.

Category	Allocation
Elements	<i>Team</i> —a group who may submit or review component changes.
	<i>Repository</i> —version control repository containing software components.
Relations	<i>Owns</i> —indicates a team responsible for reviewing changes and maintaining conceptual integrity over a repository. The owner is sometimes called the repository's benevolent dictator.
	<i>May contribute to</i> —indicates a team who may submit changes to a repository.
Rules for Use	Repositories typically have only one owner but this is not a strict rule. Teams to contribute to many repositories.
Strengths	Promotes reusability, maintainability, and development speed
Weaknesses	This pattern is strongly tied with the component partitioning strategy. In many cases the learning curve for contributions is so steep this pattern becomes impractical. May require the owning teams to adopt other open management practices to be successful.

The open source contribution pattern pairs well with any architecture patterns that also promotes reusability. Giving teams the ability to contribute changes can create opportunities for reuse that might not exist otherwise.

Big Ball of Mud Pattern

The *big ball of mud* pattern is less a pattern you choose as it is a pattern you find in the real world. The big ball of mud is described by Brian Foote and Joseph Yoder in *Pattern Languages of Program Design 4* [FHR99].

The big ball of mud pattern has no defined elements or relations. Big balls of mud don't promote any qualities in particular. As you can imagine, or have yourself experienced, big balls of mud inhibit maintainability and extensibility. Both module and module and component and connector structures can be big balls of mud. Simon Brown has observed that many microservices systems can evolve into distributed big balls of mud just as easily as monolithic systems.³

Since big balls of mud are found only in the real world, not on paper, the one positive thing we can say about them is that they promote short-term development speed at the cost of long-term design integrity. Big balls of mud often emerge due to undisciplined development practices and a general lack of understanding of the architectural principles at play in the software system.

Some big balls of mud are created on purpose to ship value sooner such as when a team strategically decides to accept technical debt in exchange for faster initial development. The dangers of this approach lay in not retiring or paying down the debt in a timely fashion.

Discover New Patterns

Patterns are born from experience. New patterns emerge every day. Some patterns might apply to a variety of systems and teams. Others might be hyper local, perhaps only applicable to a single organization. New patterns spontaneously emerge all the time, though one does not simply invent a new architecture pattern.

Architects discover patterns in much the same way an entomologist discovers a new species of insect. Spend time in the field. Observe the world around you. When you've identified a possible pattern, describe it and classify it relative to existing patterns. If your discovery is similar to something that exists,

3. http://www.codingthearchitecture.com/2014/07/06/distributed_big_balls_of_mud.html

add your knowledge to our collective wisdom of the existing pattern by publishing a blog post or paper. If your discovery is new, then add it to your team's pattern catalog.

There are two primary approaches to discovering patterns: problem focused and solution focused. With the problem-focused approach, you start by looking for a common problem. Once you've seen the same problem a few times, your goal is to develop a generalized solution. Survey the existing solutions. Look for similarities and differences among the current solutions. Based on your analysis, attempt to describe the solution pattern.

With the solution-focused approach, you start by looking for solutions that are used again and again, perhaps without developers realizing it. Describe the solution pattern as you've observed it. Do some analysis to uncover the common problem being solved and attempt to define it.

Once you have a pattern, send it out for feedback. Look for people who are familiar with the problem or have implemented the solution before. The final test of your pattern will be in its first implementation. Use the feedback from your reviewers and the early implementation attempts to improve the pattern.

Project Lionheart: The Story So Far...

The team gathered in a conference room to share findings from their recent design exploration. Choosing a search technology was easy. The group gave a short presentation of the available technologies, shared a brief demo, and recommended a technology that seemed reasonable enough. Finalizing the basic patterns in the architecture, it seems, is a different matter.

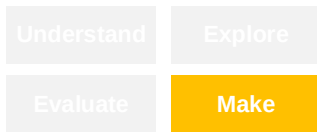
"What if we go with a simple 3-tier system?" Leia suggests. She steps up to the whiteboard and sketches some boxes and lines. An impromptu whiteboard jam ([described on page 255](#)) breaks out. Owen counters, "I've been reading about microservices. It seems like a slick solution." Owen explains the microservices pattern to the team. Finn pipes up, "Microservices sound exciting but also seem like a lot of work for a simple web application."

The whiteboard jam continues for several minutes as teammates propose new patterns and discuss the merits of each pattern in turn. "Lots of good discussions," you chime in, "but we're losing focus and need to make a decision. How do the proposed solutions influence our top-quality attributes?"

Next Up

In this chapter, we've only just scratched the surface on a topic that goes very deep. There are many, many published patterns catalogs in the software design literature. The more patterns you know, the better a software designer you will become.

As we choose patterns and make design decisions, we must share our decisions with others. In the next chapter, you'll see how to make the design tangible by identifying the essential design concepts in the architecture and creating models.



CHAPTER 8

Manage Complexity with Meaningful Models

Complexity is an inevitable by-product of every successful software system. More users will push the limits of availability, scalability, and performance. New features will be bolted on and wedged in wherever they fit. As software grows, the sheer size of the system can overwhelm the teams who develop it. Without constant vigilance, software systems eventually become victims of their own success.

All hope is not lost. When complexity rears its ugly head, we have options for keeping it in check. We can make the software smaller again by altering the requirements and snipping out code. We can divide big, complex things into smaller things that are easier to reason about and manage. We can also hide details and think about the software from the perspective of coarser-grained abstractions.

In *Define the Essential Structures*, on page 7 you learned that architecture is made up of structures, which in turn are composed of elements and relations. In this chapter you'll learn how to use these basic building blocks to create meaningful models that help us reason about our designs.

Reason About the Architecture

There is a finite amount of information we can keep in our heads at any given time. Over the years, software developers have created cheats to work around the limitations in our brains. One cheat is to turn problem solving into a massively parallel operation by collaborating with other humans. Another involves creating new, abstract concepts to represent chunks of knowledge.

These tools—collaboration and abstraction—give us what we need to think through, analyze, and understand our architectures.

Abstractions help us focus on specific details at the expense of others. For example, an interface of a class in object-oriented programming describes the public methods but says nothing about how we should implement those methods. The concrete implementation of the interface provides those details. Removing these distracting details tightens focus on what you want to think about: the interface.

Of course, the perfect abstraction is not useful if we can't share it. Anyone can draw boxes and lines. Creating a genuinely useful *model* of the architecture takes serious thought. A model, unlike any old sketch, is a precise and accurate description of some piece of the architecture that enhances communication and can be used to reason about the system.

Good architecture models have many benefits:

Models establish the vocabulary of design. Words matter. A good element name will convey meaning and intent. Each model we create extends the software system's vocabulary. We use this vocabulary in our day-to-day discussions, but it also permeates the code we write and shapes the way we see the world.

Models direct our attention to interesting details. In software development, details are everything. Just because all details are important it does not mean we want to (or have the cognitive capacity to) think about all the details at the same time. Models let us hide some details so we can focus only on what is needed at this moment to answer a specific question.

Models allow us to reason about quality attributes and other system properties. Models make it easier to think about and describe how the system would behave. If we build the right models of our architecture, we can use them to test our designs before implementing the system too. We'll still need to run experiments and build prototypes so that we can learn how to create accurate models. Even then, running an experiment is significantly cheaper and faster than learning that the design stinks after we built the whole system!

Models capture the architect's intent. All developers should understand why we designed the system the way we did. Great models express the intent behind the structures. The more people who understand this intent, the greater the chance we'll have of maintaining the system's conceptual design integrity as we evolve the system over time.

Models are born from our perception of the world and our struggle to communicate the deeper meaning behind our design intent. Every model has concepts and rules, which describe how to use those concepts. Correctly applying the rules keeps the model consistent with our perception of the world.



Joe asks:

What if I Want to Skip All This Modeling Stuff and Start Writing Code Now?

Gregor Hohpe, author of *Enterprise Integration Patterns: Designing and Deploying Messaging Solutions* [HW04] and *37 Things and Architect Knows: A Chief Architect's Journey* [Hoh16], has a saying: *A month of coding can save an hour of architecting*. We know that architecture flaws are cheaper and faster to fix while we're still designing architecture than when we're writing code, running acceptance tests, or (*gasp!*) reviewing customer complaints after release. Gregor knows that an architecture problem is much faster to fix while it's on a whiteboard than captured in thousands of lines of code.

Writing code is a fantastic way to learn about the system and how to design it. We should write code early. We should run experiments and build prototypes while we're discovering the architecture. Thinking about models is not a substitute for firsthand experience. It is impossible to predict everything about a software system only with diagrams.

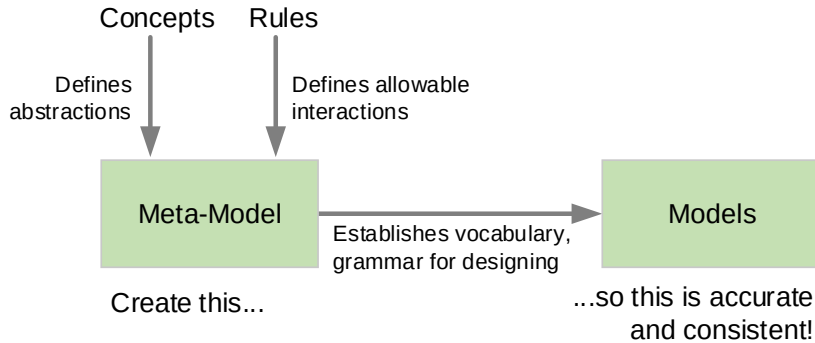
As you learned in *Decide How Much to Design Up Front*, on page 29, we'll end up with an architecture whether we design it up front, let it emerge as we write code, or do a little of both. Before you skip modeling the architecture and jump straight into writing code, think for a moment. When do you want to pay for your design and how much rework can you afford? A month of coding can save an hour of architecting.

Design the Meta-Model

A software system's *architectural meta-model* defines the concepts used in a model and the rules for how those concepts are applied. The meta-model is like a cognitive grammar for design. It constrains our thinking and creates the vocabulary we use to talk about the architecture.¹ The [diagram on page 102](#) summarizes these ideas.

Defining a meta-model makes it easier to describe the architecture, set expectations for our audience, and reason about the models we create. To create a meta-model, start by defining the concepts at play. These will be the elements and

1. George Fairbanks. *Teaching the Architecture Metamodel-First*. SATURN 2014.
<http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=89518>



relations in the architecture. Once we've described the concepts, establish the rules for using those concepts.

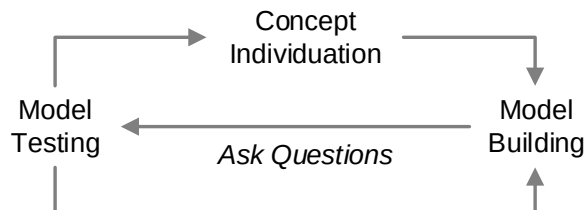
Individuate New Concepts

Concept individuation is the cognitive process by which we recognize ideas as being distinct from one another. As we individuate new ideas in the architecture, we update our understanding of the world and the models that represent it.

You have individuated concepts naturally since birth. Imagine the first time you encountered a door as a toddler. By watching adults use doors, you surmised that you could open doors by turning knobs. You individuated the concept of a door as distinct from a wall because you figured out that doors have knobs. One day you turned a knob to open a door, and it didn't budge. Through this discovery, you individuated the concept of a locked door as distinct from an unlocked door and updated your internal mental model of the world.

To individuate concepts, we follow a simple process called the [curiosity cycle](#) [Mug14]. This process applies when defining any model, whether it is a model of our world or a software system's architecture.

Mugan's Curiosity Cycle

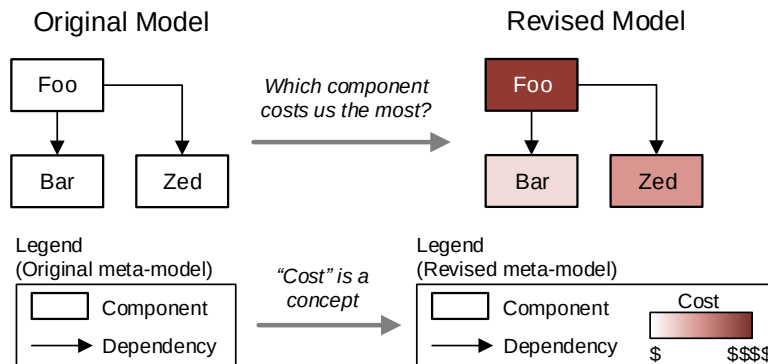


Start the process by asking a question. This question forms the basis for a test. Through the test, we'll either find the answer in our model or find a gap in our current understanding. If we find an answer, then we reinforce our existing model. When we find a gap, we must figure out how to change our model to fill it.

Here's an example of the curiosity cycle in action. Say stakeholders have prioritized availability, but we also need to keep costs down. Under these circumstances, we might ask the following questions:

- *Question:* Which components cost us the most if they are unavailable?
- *Test:* I can't answer this question with our current model. I see components, but I can't discern their costs.
- *Individuate a new concept:* Let's introduce cost.
- *Build new model:* We can use color in our meta-model to represent costs. White means no loss whereas a dark red box means we're losing a lot of money. The darker the color, the greater the loss.
- *Test:* Ah-ha! Now I can see that component Foo is going to cause the most if it is unavailable.

After updating the meta-model to include the concept of *cost*, we can now create a new model that lets us reason about the relationship between costs and other quality attributes.



It takes time to arrive at the *ah-ha!* moment. There is also a risk that we may not have the knowledge or experience necessary to individuate concepts required to design the right system. Starting with an existing meta-model, such as an architecture pattern, can reduce this risk.

Pick a Pattern to Seed the Architecture

As discussed in [The Four Principles of Design Thinking, on page 15](#), all design is redesign. Patterns are the ultimate example of the redesign rule in action. Architecture patterns describe a prepackaged meta-model relevant to a specific problem. Pick an appropriate pattern, and the meta-model is free.

We explore several popular patterns in [Chapter 7, Create a Foundation with Patterns, on page 79](#). Notice how each pattern describes the elements, relations, and rules. Each pattern's meta-model is complete, consistent, but also flexible so that designers may use the pattern under varying circumstances.

Most architectures include one or two thematic patterns to seed the system's design. Even with these thematic patterns, there are still many details to sort out. As we add new concepts to the meta-model, it is possible that could undermine the patterns in our architecture's foundation.

Reconcile Inconsistencies

Combining meta-models—for example, by merging patterns—may introduce inconsistencies in the combined meta-model. For example, two meta-models might define a *worker* element, but the responsibilities for the worker and rules for its use are radically different. Reconciling inconsistencies involves merging similar concepts and renaming different concepts with the same name so that they remain distinct. Rules may also need to be adjusted. Failure to resolve these inconsistencies can undermine our design efforts.

As we add new concepts to the meta-model, the rules for using those concepts should also be updated. Rules describe how elements and relations interact in the system. They must reflect reality. For example, many programming languages have strict type systems. If we implemented a pipe-and-filter system ([introduced on page 84](#)) using a language with strong types, then the meta-model should include rules about types. If the language we choose doesn't enforce types, then we'll need to define message descriptions or protocol headers.

Rules also describe *conceptual constraints* we choose to place on the architecture. Conceptual constraints allow us to promote specific quality attributes. For example, in the layers pattern ([introduced on page 80](#)), an element within a *layer* is only *allowed to use* other elements in the same layer or the layer directly below it. We created this rule to promote maintainability. Violate this rule, and we'll end up with an unmaintainable bowl of spaghetti.

We create rules the same way as other concepts. Ask a question. Test the model. Update the model by amending or adding rules. Repeat until the model sufficiently answers your questions.

Use Good Names

Naming things is hard. Naming things is also surprisingly important. Since naming is a design tool, many of the principles you already know from writing good code apply to architecture too.

In *Good Naming Is a Process, Not a Single Step*,² Arlo Belshee describes seven stages of naming. The names we choose reflect how well we understand what we're designing. As our understanding improves, so too do the names we give the concepts. Here are Belshee's 7 Stages of Naming applied to software architecture.

Stage 1: Missing

No name. We don't know enough about the system or context to extract a named element.

Stage 2: Nonsense

Name has absolutely no meaning. We have identified a chunk of ideas as being somehow related.

Stage 3: Honest

The name describes at least one of the element's responsibilities.

Stage 4: Honest and Complete

The name directly describes *all* of the element's responsibilities.

Stage 5: Does the Right Thing

The name reflects a conscious decision to evolve the element's responsibility. This only happens as we gain more knowledge about the element's role in the context of the architecture.

Stage 6: Intent

The name describes the element's responsibility but also its purpose. Understanding purpose requires that we understand *why* the element exists in addition to what it does.

Stage 7: Domain Abstraction

The name transcends individual elements to create a new abstraction. This is where new concepts for the meta-model are born.

2. <http://arlobelshee.com/good-naming-is-a-process-not-a-single-step/>

Here's an example of one naming progression. In this project, we were attempting to name a set of elements responsible for fetching data from a web service and transforming it.

Stage	Name
1. Missing	The thing that does the thing
2. Nonsense	Cranberry
3. Honest	Job Starter Process
4. Honest and Complete	Data Fetcher, Checker, Transformer, and Job Starter
5. Does the Right Thing	Data Transformation Job Runner
6. Intent	Data Preparer
7. Domain Abstraction	Data Preparation Agent

The final name emerged from the realization that there are several *Agents* with similar responsibilities and interaction rules across the system. The concept of *Agents* was a powerful idea that let us create clean abstractions and an improved meta-model. Now the name *Agent* carries meaning and communicates intent for architectural elements bearing that name.

Use names as a litmus test to determine how well you understand the concepts in the architecture. If your names are nonsense or simply honest, then you may have more work until you understand the concepts you're designing.

Get Your Hands Dirty: Create an Architecture Flipbook for Conway's Game of Life

Conway's Game of Life is a zero player simulation in which the universe consists of a two-dimensional grid of cells.³ For any given iteration of the game, a cell may be either alive or dead. There are four rules that determine a cell's state:

1. Any live cell with fewer than two live neighbors dies (under-population).
2. Any live cell with two or three live neighbors lives on to the next generation.
3. Any live cell with more than three live neighbors dies (over-population).
4. Any dead cell with exactly three live neighbors becomes a live cell (reproduction).

3. https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life

Individuate the concepts in a meta-model that might describe Conway's Game of Life by making an Architecture Flipbook, described on [Activity 12, Architecture Flipbook, on page 228](#).

Here are some things to think about:

- What are the nouns and verbs in the game's description and rules?
- Looking at the names you choose, where do they lie on Belshee's 7 Stages of Naming?
- What questions do you need to answer with your model?
- What elements and relations are in your diagram's legend?

Build Models into the Code

Models allow us to reason about the architecture, but they suffer from a fatal flaw. Most models are a *representation* of the architecture, inherently divorced from the code. Without care, the ideas discussed in our models will never make their way into the code we write. Alas, all our thinking and reasoning about quality attributes is for naught!

Despair not. With some thought, we can build many of our architecture models directly into the code. There are many benefits to building architecture models into the code. When the architecture is self-evident within the code, it's easier to maintain conceptual design integrity and promote desired quality attributes. Building models into the code decreases the chances of architectural drift since models and code move nearly in lock step. We also alleviate the need for some documentation since we've embedded design intent into the software system itself.

Unfortunately, in [Just Enough Software Architecture: A Risk-Driven Approach \[Fai10\]](#), George Fairbanks shows us that it is not possible to directly realize all design concepts from the architecture's conceptual meta-model in code. It is possible to shrink this *model-code gap*, but we can't close it completely.

To shrink the gap between our models and the code, we can use what Fairbanks calls an *architecturally evident coding style*. With this approach, we embed hints about our models, their rules for use, and the rationale behind the design into the code. Doing this lets us close enough of the gap to make our mental models of the architecture come alive in the code we write.